BAIL security

SECURITY AUDIT

Nest

Algebra Plugin

FINAL REPORT

Oct 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Nest – Algebra Plugin – Audit Report
Website	usenest.xyz
Language	Solidity
Methods	Manual Analysis
Github repository	old deployment: https://github.com/FenixProtocol/nest-algebra/tree/c8c8e983df304c544288df66e39d64b42a33104f new deployment: https://github.com/FenixProtocol/nest-algebra/tree/3139ba1c75e24028dda2a9d0e2602cdf0025bbc2
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	5					
Low	32					
Informational	18					
Governance	1					
Total	56					

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

This audit is focused on the upgrade to the NEST protocol, adding custom pool functionality and more plugins. The issues flagged in the last audit still apply and can be found here. This report is only focused on the update and documents any new issues since the last revision.

Core

General

Issue_01	Privileged roles have full control over the protocol
Severity	**Governance**
Description	Several privileged roles and contracts have full control over the protocol, without any delay enforced in the code: - The factory ProxyAdmin and the ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR role can upgrade the factory and all the plugins, which could be used to steal users' funds. - The factory owner can give any role, and with the POOLS_ADMINISTRATOR_ROLE can change the plugin, fees and settings of any pool. - The custom pool deployers and plugin factory owners have the same control over their own pools.
Recommendations	Consider using multisigs and timelocks for all privileged roles, and documenting what each of them can do.
Comments / Resolution	

AlgebraFactoryUpgradeable

The update adds a second pool type, custom pools, to the AlgebraFactoryUpgradeable contract. A custom pool is keyed by a custom deployer address as well as the token pair. Custom pools are registered in customPoolByPair, and deployerByPool records the custom deployer each one was created under.

The update adds a createCustomPool function, which only holders of the CUSTOM_POOL_DEPLOYER role can call. It asks the caller for a plugin through beforeCreatePoolHook, deploys the pool with the custom deployer as an extra salt input, and then calls afterCreatePoolHook. The update also changes initialize to take an explicit initial owner, and adds a setPoolDeployer function that lets the owner set a new pool deployer once after an upgrade.

Appendix: Legacy and new contract versions

The factory is upgraded in place behind its proxy, so only one instance of it exists. Several contracts around it are not upgradeable and are redeployed for the update, which leaves a legacy and a new version of each in use at the same time:

- AlgebraPoolDeployer is redeployed, and the factory is pointed at the new instance through setPoolDeployer. Pools created by the legacy deployer stay registered in the factory.
- NonfungiblePositionManager and its NonfungibleTokenPositionDescriptor are redeployed. The NonfungiblePositionManager appendix describes how the two versions differ.
- SwapRouter, Quoter and QuoterV2 are redeployed. The legacy instances keep the token-only path format and the old pool deployer.
- BasePluginV1Factory remains the legacy plugin factory for classic pools, alongside the new BaseV1PluginFactory.

The legacy periphery contracts compute pool addresses from the old pool deployer, so they work only with classic pools created before the switch. The new periphery contracts read the factory registry, so they reach classic pools of both generations and all custom pools.

Privileged Functions

- createCustomPool [CUSTOM_POOL_DEPLOYER]
- setPoolDeployer [owner]
- createPool [POOLS_CREATOR_ROLE if in private mode, else permissionless]
- setIsPublicPoolCreationMode [owner]
- setDefaultCommunityFee [owner]
- setDefaultFee [owner]
- setDefaultTickspacing [owner]
- setDefaultPluginFactory [owner]
- setVaultFactory [owner]
- startRenounceOwnership [owner]
- stopRenounceOwnership [owner]
- renounceOwnership [owner]
- transferOwnership [owner]
- acceptOwnership [pending owner]
- grantRole [DEFAULT_ADMIN_ROLE]
- revokeRole [DEFAULT_ADMIN_ROLE]
- renounceRole [role]

Invariants

- For every custom pool p, deployerByPool[p] is non-zero and customPoolByPair[deployerByPool[p]][token0][token1] equals p.
- A (customDeployer, token0, token1) triple maps to at most one custom pool.
- Classic and custom pool salts never collide because they hash a 2-word and a 3-word encoding respectively.
- setPoolDeployer can succeed at most once per proxy (reinitializer version 2).

Issue_02	Factory upgrade can not be made atomic
Severity	Low
Description	The setPoolDeployer function is gated by the onlyOwner modifier, to make sure only the owner can reinitialize the contract. However, the upgrade call must come from the ProxyAdmin, which can't own the proxy: a transparent proxy never forwards the admin calls to the implementation. This means that the upgradeAndCall function can't be used to upgrade and reinitialize at the same time, and the upgrade has to be done in 2 separate transactions. Between these 2 transactions, the new implementation calls the new deploy function (with 4 parameters) on the old pool deployer, which only has the old version (with 3 parameters). As a result, every call to createPool reverts until setPoolDeployer is called.
Recommendations	Consider removing the onlyOwner check to allow upgrading and reinitializing in the same call. In that case, and always doing the upgrade with upgradeAndCall. In that case, the initialize and setPoolDeployer function should also be merged, so that if the contract needs to be redeployed, the entire initialization flow can be done in a single call. Alternatively, if the contract is owned by a multisig, the 2 calls can be batched but this should be well documented.
Comments / Resolution	

Issue_03	setPoolDeployer doesn't check the new deployer
Severity	Low
Description	The setPoolDeployer only checks that the new address is not zero. However, nothing makes sure the new address is a pool deployer made for this factory. If a wrong address is set by mistake, pool creation is broken, and the owner can't fix it since the function can't be called again. The only way to fix it would be to upgrade to a new implementation.
Recommendations	Consider checking that the new deployer is linked to this factory, for example by checking that getDeployParameters() returns address(this) as the factory.
Comments / Resolution	

Issue_04	Custom pools can be created for any deployer
Severity	**Low**
Description	The createCustomPool function can be called by any holder of the CUSTOM_POOL_DEPLOYER role. The customDeployer parameter is used to compute the pool address and to register the pool in customPoolByPair. However, the creation hooks are called on msg.sender, and there is no check that msg.sender is the customDeployer. This means that any holder of the role can create a pool for another deployer, with any plugin. For example, it could create the pool of a pair for the Nest factory. Then: - the Nest factory can never create a pool for this pair; - the Nest factory owner can't manage this pool; - the router, quoters and position manager will use this pool for this deployer.
Recommendations	Consider only giving this role to contracts that ensure that customDeployer == msg.sender.
Comments / Resolution	

Issue_05	Custom pool registry is updated after the hooks
Severity	**Low**
Description	In createCustomPool, the customPoolByPair and deployerByPool mappings are updated after both beforeCreatePoolHook and afterCreatePoolHook are called. During the hooks, the registry is still empty, so a hook could create the same pool again and pass the registry check. Only the CREATE2 address collision would stop it. The original Algebra code uses the nonReentrant modifier on this function, which was removed here. This is fine as long as only trusted plugin factories can create pools.
Recommendations	Consider updating the registry before calling afterCreatePoolHook, and adding back the nonReentrant modifier.
Comments / Resolution	

Issue_06	Nest public pool creation bypasses the factory's pool-creation gate
Severity	**Low**
Description	createPool restricts pool creation to POOLS_CREATOR_ROLE unless the factory owner enables isPublicPoolCreationMode. However, createCustomPool only checks that the caller (the entry point) holds CUSTOM_POOL_DEPLOYER: *require(hasRole(CUSTOM_POOL_DEPLOYER, msg.sender), 'Cant create custom pools');* NestFeePluginFactory has its own switch, and when it is on, anyone can create a pool: *if (!isPublicPoolCreationMode) _checkRole(CUSTOM_POOL_DEPLOYER);* So the Nest factory owner can open pool creation to everyone while the core factory keeps it private. Since only one Nest pool can exist per pair and initialization is permissionless, the first caller takes the pair's slot and sets its starting price. Gauges still require governance.
Recommendations	Consider applying the factory's creation check to creator in createCustomPool, or documenting that the Nest switches override it and creating key pairs before enabling it.
Comments / Resolution	

Issue_07	computePoolAddress returns wrong addresses for old pools
Severity	Informational
Description	After setPoolDeployer is called, computePoolAddress uses the new pool deployer. However, the pools created before the upgrade were deployed by the old one, so the function returns a wrong address for all of them. Any integration using this function to find or check a pool will fail for these pools. Also, deployerByPool returns zero for classic pools and for unknown addresses, so it can't be used to check if an address is a pool.
Recommendations	Consider documenting that only the registry mappings should be used.
Comments / Resolution	

AlgebraPoolDeployer

The AlgebraPoolDeployer contract deploys pools through CREATE2 on behalf of the factory. Its deploy function now takes a customDeployer argument. When the argument is non-zero, the CREATE2 salt is keccak256(abi.encode(customDeployer, token0, token1)), which matches the factory's computeCustomPoolAddress. The temporary plugin, token0 and token1 values are now packed into two bytes32 cache slots, which are zeroed after deployment.

Privileged Functions

- deploy [factory]

Invariants

- _readFromCache returns exactly the (plugin, token0, token1) passed to the last _writeToCache.
- Both cache slots are zero outside an in-progress deploy call.

Issue_08	Wrong init code hash silently breaks pool creation
Severity	**Informational**
Description	The factory and the periphery use POOL_INIT_CODE_HASH to compute pool addresses. Since the pool is now deployed by a new pool deployer, any change in the compiler settings changes the pool bytecode, which makes this value wrong. In that case, NestFeePluginFactory saves the pending pool under the computed address, but the hooks receive the real one, so every deployCustomPool call reverts. For classic pools, the plugin is created for the wrong address and the pool can never be initialized.
Recommendations	Consider checking that keccak256(type(AlgebraPool).creationCode) == POOL_INIT_CODE_HASH in the deployer constructor or at least in the deployment script.
Comments / Resolution	

Periphery

AlgebraCustomPoolEntryPoint

The AlgebraCustomPoolEntryPoint contract is the single gateway between custom pool deployers and the Algebra factory. It holds the factory's CUSTOM_POOL_DEPLOYER role and an owner-managed allowlist of custom deployer contracts. Only allowlisted deployers can create custom pools through it.

To the factory, the entry point acts as the plugin factory. The factory calls its pool-creation hooks, and the entry point forwards each hook to the custom deployer named in the call. To pools, the entry point acts as a pools administrator.

createCustomPool requires the caller to be the deployer argument and to be allowlisted. beforeCreatePoolHook and afterCreatePoolHook accept calls only from the factory. setTickSpacing, setPlugin, setPluginConfig, and setFee forward to a pool only when the caller is the custom deployer under which the factory registered that pool. setCustomPoolDeployer and setCustomPoolDeployerBatch manage the allowlist. createPlugin always reverts.

Privileged Functions

- createCustomPool [allowlisted deployer, caller must be the deployer]
- setCustomPoolDeployer [owner]
- setCustomPoolDeployerBatch [owner]
- beforeCreatePoolHook [factory]
- afterCreatePoolHook [factory]
- setTickSpacing [custom deployer of the pool]
- setPlugin [custom deployer of the pool]
- setPluginConfig [custom deployer of the pool]
- setFee [custom deployer of the pool]
- transferOwnership [owner]
- acceptOwnership [pending owner]
- renounceOwnership [owner]

Invariants

- Pool admin calls reach a pool only if the caller is the custom deployer under which the factory registered that pool
- The plugin that the custom deployer returns from beforeCreatePoolHook is the plugin the pool is deployed with.

Issue_09	A removed custom deployer keeps admin rights over its existing pools
Severity	Low
Description	setCustomPoolDeployer(deployer, false) stops a deployer from creating new pools. However, the admin wrappers (setPlugin, setPluginConfig, setFee, setTickSpacing) only check that the caller is the pool's registered deployer: *require(pool == IAlgebraFactory(factory).customPoolByPair(msg.sender, token0, token1), 'Only deployer');* In practice, the deployer is the Nest factory, whose wrappers are controlled by its owner. So if that owner key is compromised and the deployer is removed, it can still install a malicious plugin or change fees on all its pools. Since plugin hooks run outside the pool lock, such a plugin can trade around every swap, charge up to 6.55%, revert swaps and burns, or steal funds from the position manager (see "collect, decreaseLiquidity, and burn are not protected against reentrancy"). A core administrator can revert each change, but the deployer can redo it. The only way to fully stop it is to revoke the entry point's admin role, which affects every custom deployer. Also, during an entry point migration, both entry points hold the admin role, so a deployer can manage its pools through the other entry point, even if it was never allowlisted there.
Recommendations	Consider also checking isCustomPoolDeployer[msg.sender] in _checkIfDeployer, and only accepting admin calls from the entry point that created the pool..
Comments / Resolution	

NonfungiblePositionManager

The NonfungiblePositionManager contract wraps liquidity positions as NFTs. It now supports positions in custom pools. A deployer field is added to MintParams, AddLiquidityParams, and the cached PoolKey, and positions() returns it as a new fifth field. Pools are now resolved from the factory registry instead of by CREATE2 address computation. The mint and increaseLiquidity functions are also nonReentrant now.

Appendix: Legacy and new deployments

The NonfungiblePositionManager contract is not upgradeable, and the update changes the mint ABI and the positions return layout. The updated contract is therefore deployed as a second position manager next to the live one, and both stay in use. Existing positions remain in the legacy position manager and are not migrated. The new position manager starts again at tokenId 1 and reuses the same NFT name and symbol.

The two position managers reach different pools. The legacy one computes pool addresses from the pool deployer it was constructed with, so it works only with classic pools created before the factory switches to the new pool deployer. The new one reads the factory registry, so it reaches classic pools of both generations and all custom pools.

Each position manager needs its own NonfungibleTokenPositionDescriptor proxy. A descriptor implementation decodes one positions() layout, either the legacy 11 fields or the new 12 fields, so a single proxy cannot serve both position managers.

The AlgebraFactoryUpgradeable appendix lists the other contracts that exist in a legacy and a new version.

Privileged Functions

- setFarmingCenter [NONFUNGIBLE_POSITION_MANAGER_ADMINISTRATOR_ROLE or factory owner]
- switchFarmingStatus [farming center; also the farming center the NFT is farmed in when switching off]

Invariants

- Positions in a classic pool and a custom pool for the same token pair get different poolIds.

Issue_10	collect, decreaseLiquidity, and burn are not protected against reentrancy
Severity	**Medium**
Description	The update adds a shared reentrancy guard to the position manager, on mint, increaseLiquidity, and the payment functions. However, decreaseLiquidity, collect, and burn are not guarded, so the other functions can still be called while they run. During collect, the pool transfers token0 and then token1. A malicious token can use this to call sweepToken, unwrapWNativeToken, or refundNativeToken and take what the position manager holds in the middle of the user's multicall. This includes tokens collected to the position manager before an unwrap or sweep, even token0 of the same pool, and the msg.value of a later native-funded mint, which then takes the missing amount from the user's WNative allowance. The user only loses funds if the later unwrapWNativeToken or sweepToken uses a zero amountMinimum. Otherwise, the multicall reverts. Also, increaseLiquidity, decreaseLiquidity, and collect cache the position's liquidity or owed amounts before calling the pool, and write them back after: *uint128 positionLiquidity = position.liquidity;* *...* *position.liquidity = positionLiquidity + liquidity;* The pool runs the plugin's position hooks outside its lock, and all NFTs with the same ticks share one pool position. So a malicious plugin can call decreaseLiquidity from inside increaseLiquidity, and the outer call writes back the liquidity that was just withdrawn. The guard on increaseLiquidity doesn't help, since decreaseLiquidity is not guarded.
Recommendations	Consider adding the nonReentrant modifier to decreaseLiquidity, collect, and burn, which fixes both cases. Integrations should also always use a non-zero amountMinimum in unwrapWNativeToken and sweepToken.
Comments / Resolution	

Issue_11	Third-party liquidity increases can erase farming rewards during emergency withdrawal
Severity	Medium
Description	Anyone can call increaseLiquidity on a farmed NFT. During emergency withdrawal, the resulting farming update exits the position and deletes its farm without crediting accrued rewards. A third party can therefore add dust liquidity to another user's position, erase its uncollected rewards and remove it from farming. Previously credited rewards remain unaffected.
Recommendations	Consider preventing third-party increases from forcing emergency exits, or preserving accrued rewards when deleting the farm.
Comments / Resolution	

Issue_12	mint accepts tokens in the wrong order
Severity	Low
Description	In the previous version, mint computed the pool address from the tokens, which reverted if token0 was not lower than token1. The update now sorts the tokens and finds the pool through the factory, which works with both orders. As a result, a call with the tokens in the wrong order doesn't revert anymore. amount0Desired and amount0Min are then used for the pool's token0, which is the token the user gave as token1. The user can deposit less than expected, and the slippage check is done on the wrong token.
Recommendations	Consider adding back the check require(params.token0 < params.token1) in mint.
Comments / Resolution	

Issue_13	Changing the farming center can strand rewards after an NFT is burned
Severity	Low
Description	setFarmingCenter changes the current farming center without updating NFTs already deposited in the previous center. However, _applyLiquidityDeltaInFarming only forwards liquidity changes when the NFT's recorded center matches the current one. If the administrator replaces or clears the center while positions are still farmed, their withdrawals are not reported to the old farm. The owner can then remove all liquidity, collect the tokens and burn the NFT. burn only checks that liquidity and owed tokens are zero; it does not require the position to exit farming first. The old farm therefore keeps the withdrawn liquidity in its reward accounting. Once the NFT is burned, exiting the farm or collecting its rewards reverts because the farming center checks ownership of an NFT that no longer exists. While the old incentive remains active and the stored range is in range, rewards continue accruing to this unclaimable position, reducing the share received by other farmers. For example, if the burned position represents 10% of the active farm liquidity, 10% of subsequent rewards remain assigned to it. LP principal remains withdrawable, but the orphaned rewards are stranded
Recommendations	Consider exiting or migrating existing farms before changing the farming center. Prevent burning an NFT while tokenFarmedIn[tokenId] is nonzero, and ensure liquidity changes remain synchronized with the center holding its deposit.
Comments / Resolution	

Issue_14	Farming-center rotation leaves withdrawn liquidity earning rewards
Severity	**Low**
Description	setFarmingCenter changes the current center without migrating existing deposits. _applyLiquidityDeltaInFarming then skips positions registered in the old center, so owners can withdraw their liquidity while the old farm keeps its reward weight. If the shared farming engine is also pointed to the new center, these deposits cannot exit: the old center is no longer authorized, and the new center has no deposit record. However, reward collection through the new center remains possible, allowing owners to earn rewards on withdrawn liquidity and dilute other farmers.
Recommendations	Consider exiting or migrating existing deposits before rotating either center pointer, and keeping liquidity changes synchronized with outstanding farms.
Comments / Resolution	

Issue_15	positions() return values changed
Severity	**Informational**
Description	positions() now returns 12 values instead of 11, with the new deployer value at index 4. The mint parameters also changed. Contracts already deployed that read positions, like the farming center and the position descriptor, use the old format and the old position manager.
Recommendations	Consider deploying new versions of these contracts.
Comments / Resolution	

Issue_16	Dust liquidity can make a close-and-burn multicall revert
Severity	**Informational**
Description	Anyone can call increaseLiquidity on any position, while burn requires it to be empty: *require(position.liquidity | position.tokensOwed0 | position.tokensOwed1 == 0, 'Not cleared');* An attacker can front-run a [decreaseLiquidity, collect, burn] batch with a dust deposit, so burn reverts. The owner only loses gas and can retry.
Recommendations	Consider leaving burn out of close batches in the front end.
Comments / Resolution	

NonfungibleTokenPositionDescriptor

The NonfungibleTokenPositionDescriptor contract generates the token URI for position NFTs. It now reads the new deployer field from positionManager.positions and resolves the pool from the factory registry instead of computing it from the pool deployer.

SwapRouter

The SwapRouter contract executes single-hop and multi-hop swaps against Algebra pools. A new path format lets it route through custom pools. Each hop is encoded as tokenIn, deployer, tokenOut, where a zero deployer selects the classic pool. The router now resolves pools from the factory registry, and algebraSwapCallback validates the caller against that registry. The five external swap functions are now nonReentrant.

Invariants

- A swap callback is honored only if msg.sender equals the registry pool for the (deployer, tokenIn, tokenOut) decoded from the callback path.

Issue_17	Partial multi-hop swaps leave intermediate tokens open to theft
Severity	**Low**
Description	exactInput holds each intermediate hop's output in the router to fund the next hop. If a later pool exhausts its liquidity, it can consume only part of that input. The router checks only the final amountOutMinimum and does not refund the unused intermediate tokens. The swap can therefore succeed while leaving tokens that anyone can take through sweepToken. A liquidity provider can induce this by withdrawing downstream liquidity before execution.
Recommendations	Consider requiring full consumption of intermediate inputs, or refunding unused intermediate tokens within the same transaction.
Comments / Resolution	

LiquidityManagement

The LiquidityManagement contract adds liquidity to pools and pays the tokens owed in the mint callback. AddLiquidityParams and MintCallbackData gain a deployer field. The contract now resolves the pool through the factory registry, and algebraMintCallback checks the caller against that registry.

Invariants

- algebraMintCallback pays only to a pool that the factory registers for the decoded (deployer, token0, token1).

PeripheryPayments

The PeripheryPayments contract provides the token and native-currency payment helpers used by the periphery. It now inherits OpenZeppelin ReentrancyGuard and applies nonReentrant to unwrapWNativeToken, sweepToken, and refundNativeToken. SwapRouter and NonfungiblePositionManager inherit this guard, so all of their guarded functions share one lock.

Issue_18	Funds left in the router or position manager can be taken by anyone
Severity	**Informational**
Description	refundNativeToken, sweepToken, and unwrapWNativeToken send the contract's whole balance to the caller, and pay uses any leftover native balance for the current caller. So excess msg.value or tokens not reclaimed in the same multicall can be taken by the next caller. This only happens through user or front-end error.
Recommendations	Consider documenting that integrators must include the refund, sweep, or unwrap call in the same multicall.
Comments / Resolution	

PeripheryPaymentsWithFee

The PeripheryPaymentsWithFee contract extends the payment helpers with variants that take a fee. Its unwrapWNativeTokenWithFee and sweepTokenWithFee functions are now nonReentrant. They share the lock inherited from PeripheryPayments.

SelfPermit

The SelfPermit contract lets a user approve tokens through a permit signature in the same transaction as another call. It now inherits ReentrancyGuard, and all four permit entry points are nonReentrant. The permit calls move into private helpers so that the IfNecessary variants do not re-enter a nonReentrant function.

Issue_19	Front-running a permit reverts the user's whole multicall
Severity	**Low**
Description	selfPermit and selfPermitAllowed call the token's permit and require it to succeed: *IERC20Permit(token).permit(msg.sender, address(this), value, deadline, v, r, s);* However, a permit signature can be submitted by anyone. An observer can copy it from a pending multicall and call token.permit first. The allowance is granted, but the user's selfPermit then fails on the used nonce, and the whole multicall, including the swap or mint, reverts. The IfNecessary variants avoid this, but they are opt-in. The same applies to the position NFTs. ERC721Permit.permit is permissionless, so a copied signature consumes the nonce first, and any call that bundles the user's permit with a follow-up action reverts. There is no IfNecessary variant for it. No funds are lost, and the user can retry without the permit.
Recommendations	Consider wrapping the permit calls in a try/catch and only reverting if the allowance or approval is still missing, or documenting that front ends should use the IfNecessary variants.
Comments / Resolution	

Quoter

The Quoter contract returns swap quotes by simulating a swap and reverting. It adopts the deployer-aware path format, and it resolves and validates pools through the factory registry. The storage variable amountOutCached is removed. For exact-output quotes without a price limit, the requested amountOut is now appended to the callback path, and algebraSwapCallback requires the full amount to be received.

Invariants

- A no-limit exact-output quote reverts unless the pool delivers exactly the requested amountOut.
- The callback quotes only for a pool that the factory registers for the decoded hop.

Issue_20	Quotes can be faked by plugins and tokens
Severity	**Informational**
Description	The quoters simulate a swap and read the result from the revert data. Any revert of 64 bytes (Quoter) or 224 bytes (QuoterV2) is used as a valid quote. Since the plugin hooks and the token transfers run during the swap, a malicious plugin or token can revert with fake data and return any amount. This also skips the new check that the full output amount is received. Swaps still check their own slippage limits, so users don't lose funds directly, but contracts using these quotes on-chain could be affected.
Recommendations	Consider documenting that quotes can only be trusted for pools with trusted plugins and tokens.
Comments / Resolution	

QuoterV2

The QuoterV2 contract returns swap quotes together with gas and tick-crossing estimates. It receives the same changes as Quoter: the deployer-aware path format, registry-based pool lookup and validation, and an amount appended to the callback path in place of amountOutCached.

Issue_21	Path and callback-validation changes break integrators silently
Severity	**Informational**
Description	A path is now token, deployer, token, ..., but the multi-hop natspec in IQuoter and IQuoterV2 still says "each token pair", and ISwapRouter does not describe the format. An old-format path decodes to wrong addresses and fails with an unclear error. Similarly, CallbackValidation.verifyCallback keeps its name and address first parameter, but it now expects the factory instead of the pool deployer. External code that still passes the deployer compiles fine and reverts on every callback. The library also now has five functions doing the same registry lookup: three verifyCallback overloads, verifyCallbackFromFactory, and verifyCustomCallbackFromFactory.
Recommendations	Consider updating the natspec to describe the new path format, including address(0) for classic pools. Consider also renaming the CallbackValidation functions so stale callers fail to compile, and keeping a single API.
Comments / Resolution	

CallbackValidation

The CallbackValidation library checks that a callback comes from a genuine Algebra pool. The check now reads the factory registry instead of computing a CREATE2 address, and deployer-aware variants are added. Every variant requires msg.sender to equal the resolved pool. A pair that is not registered resolves to address(0), so no caller can pass the check for it.

Path

The Path library decodes the byte-encoded swap paths used by the router and quoters. The hop encoding changes from token, token to token, deployer, token. The offsets, decodeFirstPool and skipToken are updated to match the new 40-byte hop size.

PoolAddress

The PoolAddress library builds pool keys and derives pool addresses for the periphery. PoolKey gains a deployer field, and a deployer-aware getPoolKey overload is added. computeAddress now uses the 3-word salt encoding for keys with a non-zero deployer. A new getPool function reads the pool from the factory registry and is now the main lookup used across the periphery.

Issue_22	poolDeployer is misleading after the deployer change
Severity	**Low**
Description	PoolAddress.computeAddress is not used anymore and returns wrong addresses for old pools. The periphery poolDeployer value is also not used by the periphery. However, FarmingCenter reads it from the position manager to check pools, so the deployer given to the position manager decides which pools can be farmed.
Recommendations	Consider removing computeAddress, and documenting which deployer should be given to the position manager.
Comments / Resolution	

PositionValue

The PositionValue library computes the token amounts and fees owed to a position. It follows the new positions() layout, which includes the deployer field, and it now resolves the pool through PoolAddress.getPool instead of computing it from the pool deployer.

Plugins

BasePluginV1Factory

The BasePluginV1Factory contract deploys AlgebraBasePluginV1 plugins for classic pools. It gains beforeCreatePoolHook and afterCreatePoolHook to match the extended IAlgebraPluginFactory interface. Both always revert, so this factory cannot be used as a custom pool deployer.

NestFeePluginFactory

The NestFeePluginFactory contract is the abstract base for the Nest plugin factories. It deploys one plugin per pool as a BeaconProxy that points to the factory itself, so one call to upgradeTo changes the logic of every plugin it has deployed. It covers plugins for new classic pools, for existing classic pools, and for custom pools that it deploys itself.

Access is split three ways. The owner administers custom pools and controls public creation mode. Holders of ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR on the Algebra factory (or the Algebra factory owner) control the default deviation config, the farming address, and upgrades. Holders of CUSTOM_POOL_DEPLOYER on this factory may create custom pools when public creation mode is off.

deployCustomPool stores a pending record under the predicted pool address and calls the entry point. beforeCreatePoolHook checks the caller's data against that record and creates the plugin. afterCreatePoolHook checks the plugin, deletes the record, and sets isCustomPool. deployCustomPool reverts unless the resulting pool matches the predicted address and was finalized. setTickSpacing, setPlugin, setPluginConfig, and setFee let the owner administer finalized custom pools through the entry point.

Privileged Functions

- createPlugin [Algebra factory]
- createPluginForExistingPool [POOLS_ADMINISTRATOR_ROLE or Algebra factory owner]
- deployCustomPool [CUSTOM_POOL_DEPLOYER if in private mode, else permissionless]
- beforeCreatePoolHook [entry point]
- afterCreatePoolHook [entry point]
- setTickSpacing [owner]
- setPlugin [owner]
- setPluginConfig [owner]
- setFee [owner]
- setPublicPoolCreationMode [owner]
- setDefaultDeviationFeeConfiguration [ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR or Algebra factory owner]
- setFarmingAddress [ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR or Algebra factory owner]

- upgradeTo [ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR or Algebra factory owner]
- grantRole [DEFAULT_ADMIN_ROLE]
- revokeRole [DEFAULT_ADMIN_ROLE]
- renounceRole [role]
- transferOwnership [owner]
- acceptOwnership [pending owner]
- renounceOwnership [owner]
-

Invariants

- pluginByPool[pool] is written at most once per pool.
- isCustomPool[pool] becomes true only after both hooks run against a matching pending record created by deployCustomPool in the same transaction.
- No pending record remains after a successful deployCustomPool.
- defaultDeviationFeeConfiguration.scalingFactor is always zero, so new Nest TWAP and MEV plugins start with only the base fee.

Issue_23	Connecting an incentive before the oracle is seeded halts trading
Severity	Medium
Description	A plugin attached to a pool that is already trading does not record prices until an administrator calls initialize(). Until that call, isInitialized is false. AlgebraPool.setPlugin also leaves every hook turned off, so swaps continue without calling the plugin. setIncentive skips that check. The configured farmingAddress can connect a virtual pool as soon as the plugin is attached. Because the plugin is the pool's current plugin, setIncentive calls _updatePluginConfigInPool. That call replaces the pool's hook settings with defaultPluginConfig. Those settings include the before-swap hook, and a connected incentive also turns on the after-swap hook. The pool then calls beforeSwap on every swap. beforeSwap calls _writeTimepoint, and _writeTimepoint reverts with Not initialized while the oracle is empty. The swap reverts with it. A quote reverts for the same reason, because the quoter runs pool.swap. Adding and removing liquidity still succeeds, because beforeModifyPosition does not read the oracle. Removing the incentive does not turn the before-swap hook off. setIncentive(address(0)) writes defaultPluginConfig again, so that hook stays on. Swaps start working again after the plugin factory or a pools administrator calls plugin.initialize(). A pools administrator can also restore swaps immediately by calling setPluginConfig(0). NestMEVFeePlugin uses these same functions. **Example** 1. A pool is already trading. A pools administrator creates the plugin and attaches it. The hook settings are off and isInitialized is false. A swap of 100 token0 succeeds. 2. farmingAddress calls setIncentive with a virtual pool. 3. The pool's hook settings change to the plugin default, so the next swap must call beforeSwap. 4. A swap of 100 token0 reverts with Not initialized. A quote for that same swap reverts too.

	5. The liquidity provider can still burn the position. Further swaps keep reverting until plugin.initialize() records the first price.
Recommendations	In setIncentive, require isInitialized before updating the pool's hook settings. Connecting an incentive should revert until plugin.initialize() has recorded the first price.
Comments / Resolution	

Issue_24	Plugins of custom pools can't be replaced
Severity	**Low**
Description	A plugin can only be created once per pool, because _createPlugin requires pluginByPool[pool] to be empty. Also, createPluginForExistingPool only works with classic pools. This means that if the plugin of a custom pool is replaced, for example to fix an issue, no Nest plugin can be used on this pool again, since a Nest plugin only works if the factory points to it. In that case: - a new Nest plugin can't be initialized, and pool.fee() reverts while it is attached - the old plugin oracle data can't be read anymore - the fee configuration can't be changed while the oracle is enabled
Recommendations	Consider supporting custom pools in createPluginForExistingPool, and allowing the owner to update pluginByPool when the plugin changes.
Comments / Resolution	

Issue_25	Price of a new custom pool can be front-run
Severity	Low
Description	Custom pools are created without a price, and the position manager helper to create and initialize a pool only works for classic pools. The creator has to call initialize on the pool in a separate transaction, so anyone can call it first with another price. The plugin oracle also starts at this price. The creator's slippage check prevents adding liquidity at a bad price. However, for custom pools created by BaseV1PluginFactory, the base plugin oracle keeps the attacker's price. Since there is no liquidity in the pool yet, the attacker can move the price back for free, and the adaptive fee then stays at its maximum (1.5%) for about a day.
Recommendations	Consider allowing initializing the custom pool in the same transaction.
Comments / Resolution	

Issue_26	The deployer keeps CUSTOM_POOL_DEPLOYER after ownership is transferred
Severity	Low
Description	The constructor grants both DEFAULT_ADMIN_ROLE and CUSTOM_POOL_DEPLOYER to the deployer. However, _transferOwnership only moves the admin role: *_revokeRole(DEFAULT_ADMIN_ROLE, owner());* *super._transferOwnership(newOwner);* *if (newOwner != address(0)) _grantRole(DEFAULT_ADMIN_ROLE, newOwner);* So after the handover, the old deployment key can still call deployCustomPool in private mode and take any pair's only Nest slot, while the new owner cannot until it grants itself the role. The new owner can revoke the stale role through DEFAULT_ADMIN_ROLE, but only if it notices it. Other admins added by the old owner also keep their roles.
Recommendations	Consider also moving CUSTOM_POOL_DEPLOYER in _transferOwnership, and reviewing all role holders at handover.
Comments / Resolution	

Issue_27	initialize overwrites settings applied to a custom pool before initialization
Severity	Low
Description	deployCustomPool does not initialize the pool, but the pool's admin setters already work. So the Nest owner can call setTickSpacing, setFee or setPluginConfig on the uninitialized pool. However, initialize then replaces these with the factory defaults: *_setFee(_fee);* *_setTickSpacing(_tickSpacing);* The plugin's hooks also reset the plugin config and fee. Since anyone can call initialize, the owner's settings can be wiped at any time without a revert, and LPs may mint on the default tick grid instead of the intended one. An admin can reapply the settings afterwards.
Recommendations	Consider initializing the pool inside deployCustomPool, or rejecting the wrappers for uninitialized pools.
Comments / Resolution	

Issue_28	Static admin fees can exceed the Nest fee cap
Severity	**Low**
Description	The Nest plugins cap fees at MAX_FEE_CAP = 50_000 (5%). However, with DYNAMIC_FEE off, the pool accepts any uint16 fee from an admin, and the wrappers forward it unchanged: *if (isDynamicFeeEnabled) revert dynamicFeeActive();* *_checkIfAdministrator();* *...* *_setFee(newFee);* So the Nest owner or a pools administrator can set a static fee of up to 6.5535%. This requires a privileged account, and the fee returns to the plugin's control once the hooks are restored.
Recommendations	Consider enforcing MAX_FEE_CAP in the NestFeePluginFactory.setFee wrapper, or documenting that it does not apply to static fees.
Comments / Resolution	

Issue_29	Owner setFee reverts after a Nest pool is initialized
Severity	**Informational**
Description	The factory documents owner setFee as the way to change the static fee of a custom pool. On a Nest pool that call reverts as soon as the pool has been initialized, because the Nest plugin has already turned dynamic fees on. During initialize, the pool calls the plugin's beforeInitialize before it finishes its own setup. The Nest plugin uses that hook to write its config, and that config includes DYNAMIC_FEE. With that flag set, the pool accepts setFee from the plugin only. Any other caller gets dynamicFeeActive. The factory owner does not call the pool directly. NestFeePluginFactory.setFee goes through the custom-pool entry point, and the pool sees the entry point as the caller. That address is not the plugin, so the call reverts. The plugin can still change the fee later, from afterInitialize and from each swap. The pool does get an initial fee. initialize writes the pool's default fee, and afterInitialize then replaces it with the plugin base fee. After that, the owner setFee path stays closed for as long as DYNAMIC_FEE remains set.
Recommendations	Document that owner setFee works only while dynamic fees are off. For a Nest pool, point operators to setDeviationFeeConfig.
Comments / Resolution	

Issue_30	A same-block tick move can lower the next fee
Severity	Informational
Description	The comment on NestTWAPFeePlugin._updateDeviationFee says a tick move from an earlier swap in the same block raises the fee for the next one. That function runs from beforeSwap and calls _getDeviationFee, which uses the absolute distance from the current tick to the TWAP. A move away from the TWAP raises the next fee. A move back toward the TWAP lowers it. The next swap sees whichever tick the previous swap left behind, because beforeSwap recalculates before the new swap runs.
Recommendations	On _updateDeviationFee, say the next swap is charged on the absolute distance from the TWAP after the earlier swap, whether that distance grew or shrank.
Comments / Resolution	

Issue_31	isTwapReady is false when the plugin is inactive
Severity	Informational
Description	The NatSpec on INestTWAPFeeManager.isTwapReady says the function is true when stored timepoint history covers the window. In NestTWAPFeePlugin.isTwapReady, that age check is the last step. Before it, the function returns false when isInitialized is false, or when _isCurrentPlugin is false. _isCurrentPlugin fails if this contract is no longer pool.plugin(), or if pluginByPool on the factory no longer points at it. The timepoints can still be in storage in either case.
Recommendations	Say that isTwapReady also requires an initialized plugin that is still the current plugin for the pool.
Comments / Resolution	

Issue_32	Registry writes and forwarded admin calls emit no events
Severity	Informational
Description	Writes to pluginByPool and isCustomPool emit no events, and the entry point and Nest wrappers do not record the initiating deployer. The pool's own events always show the entry point as caller. So monitoring cannot easily tell which plugin is current or who started a change.
Recommendations	Consider emitting events for registry writes and forwarded admin calls.
Comments / Resolution	

Issue_33	upgradeTo accepts a plugin implementation of the wrong type
Severity	Informational
Description	_setImplementation only checks that the new implementation has code. Pointing a Nest factory at the base plugin, or the reverse, breaks plugin creation and makes existing plugins decode their fee slot in the wrong format, leading to wrong fees or reverting swaps. Switching between the TWAP and MEV implementations silently changes the fee model, and an MEV → TWAP → MEV round trip can re-arm plugins with unchecked settings. Since NestTWAPFeePlugin.defaultPluginConfig also enables BEFORE_POSITION_MODIFY, although its hook does nothing, a broken implementation also makes every mint and burn revert, so LPs can't withdraw until the upgrade is reverted. This requires an admin mistake.
Recommendations	Consider checking the plugin type in _setImplementation, and removing the unused BEFORE_POSITION_MODIFY and BEFORE_FLASH flags from defaultPluginConfig.
Comments / Resolution	

BaseV1PluginFactory

The BaseV1PluginFactory contract is a NestFeePluginFactory variant that deploys AlgebraBasePluginV1 adaptive-fee plugins instead of Nest deviation-fee plugins. It reuses the shared factory's custom-pool handshake, beacon upgrades, and admin forwarding. Its _createPlugin override applies its own defaultFeeConfiguration, which setDefaultFeeConfiguration validates and replaces. The inherited defaultDeviationFeeConfiguration is unused by this factory's plugins.

Privileged Functions

- setDefaultFeeConfiguration [ALGEBRA_BASE_PLUGIN_FACTORY_ADMINISTRATOR or Algebra factory owner]

Issue_34	Base plugin fee is not updated within the same second
Severity	**Low**
Description	BaseV1PluginFactory deploys the existing AlgebraBasePluginV1 for new and existing pools. This plugin only updates the fee when it writes a new oracle point, which happens at most once per second. However, this means that a swap in the same second as an initialization or a configuration change still uses the old fee. The Nest plugins fixed this by updating the fee on every swap, but the base plugin used by BaseV1PluginFactory didn't get this fix.
Recommendations	Consider using the same fee update logic as the Nest plugins.
Comments / Resolution	

Issue_35	BaseV1PluginFactory has an unused fee configuration
Severity	Low
Description	BaseV1PluginFactory inherits setDefaultDeviationFeeConfiguration from the Nest factory, but never uses this configuration when creating base plugins.
Recommendations	Consider disabling this function in BaseV1PluginFactory, or documenting that it has no effect.
Comments / Resolution	

NestTWAPFeePluginFactory

The NestTWAPFeePluginFactory contract is a concrete NestFeePluginFactory with no added logic. It deploys beacon-proxy NestTWAPFeePlugin instances for classic, existing, and custom pools.

Issue_36	A detached Base V1 plugin still quotes a live TWAP
Severity	**Low**
Description	A plugin is recorded in two places. The pool stores the plugin that serves its swaps. The factory stores the same plugin in pluginByPool, which external readers use to find the oracle. setPlugin updates only the pool. It clears the pool's plugin config and saves the new plugin there. pluginByPool still returns the old Base V1 proxy, and that proxy still serves getTimepoints and getCurrentFee. A TWAP read past the last stored timepoint has to fill the missing time. The proxy fills it with the pool's tick at the moment of the query. While the plugin is attached, that fill covers only the short gap since the last swap. After detachment, the proxy writes no further timepoints, so the fill covers the whole period since the last write. The quote treats today's tick as the price for that entire period. getCurrentFee then calculates its volatility fee from that same quote. The pool's own swaps use the plugin now installed on the pool. A caller that looks up the oracle through pluginByPool, or that calls the old proxy directly, receives the filled quote. Nest TWAP and Nest MEV revert these reads once they are no longer the pool's plugin. The oracle lens returns this filled tick and also reports that the oracle is disconnected. **Example** The last stored timepoint is at time 1,000, at tick 0. The plugin is detached then. The pool stays at tick 0 until time 1,500, then moves to tick 200. At time 1,600 a caller asks the old proxy for the 600-second average, covering time 1,000 to 1,600. The real average over that window is about 33: tick 0 for 500 seconds and tick 200 for 100 seconds. The proxy has no record after time 1,000,

	so it fills all 600 seconds at the current tick of 200 and returns 200. getCurrentFee prices its fee from that quote of 200.
Recommendations	Clear pluginByPool when the pool's plugin changes. Reject timepoint and fee reads on a plugin that is no longer pool.plugin(). Leave the missing time unfilled once the plugin has stopped recording.
Comments / Resolution	

Issue_37	Base V1 plugins ignore the deviation fee default
Severity	**Low**
Description	BaseV1PluginFactory inherits defaultDeviationFeeConfiguration and its setter. The shared interface says that configuration is applied to plugins at creation. The base V1 create path never reads it. A new base plugin receives defaultFeeConfiguration only, the adaptive-fee schedule whose base fee is 100 (0.01%). An administrator can change the deviation default, see the event, and still deploy base plugins on the old adaptive schedule. Nest TWAP and MEV factories use the deviation default. This factory does not. **Example:** The inherited default is base fee 500, cap 10_000, scaling 0, window 600. An admin sets the deviation base fee to 300 and the window to 120. The next base plugin is still initialized with the adaptive configuration and base fee 100.
Recommendations	Do not expose the deviation default on the base V1 factory, or do not inherit a setter the create path does not use.
Comments / Resolution	

NestTWAPFeePlugin

The NestTWAPFeePlugin contract is an Algebra pool plugin that charges a dynamic fee based on how far the current tick deviates from the pool's own TWAP. The fee is min(baseFee + scalingFactor × |tick – twapTick| / 1e6, feeCap), and it falls back to baseFee while the stored history is shorter than the TWAP window. The plugin has no external price dependency. It records a 65536-slot timepoint ring buffer, written at most once per second, and supports a farming incentive.

beforeSwap writes a timepoint and recalculates the fee on every swap, so a tick move earlier in a block raises the fee for later swaps in the same block. afterInitialize seeds the oracle, and the no-argument initialize, callable by the plugin factory or a pools administrator, does the same when the plugin is attached to an already-initialized pool. setDeviationFeeConfig is restricted to the plugin factory and to holders of the DEVIATION_FEE_MANAGER role (or the owner) on the Algebra factory. A non-zero scalingFactor requires fee headroom and a TWAP window already covered by history. setIncentive connects or disconnects a farming virtual pool.

Privileged Functions

- initialize() [plugin factory or POOLS_ADMINISTRATOR_ROLE]
- initialize(pool, factory, pluginFactory) [plugin factory, once]
- setDeviationFeeConfig [plugin factory, DEVIATION_FEE_MANAGER or Algebra factory owner]
- setIncentive [farming address to connect, last incentive owner to disconnect]
- beforeInitialize [pool]
- afterInitialize [pool]
- beforeModifyPosition [pool]
- afterModifyPosition [pool]
- beforeSwap [pool]
- afterSwap [pool]
- beforeFlash [pool]
- afterFlash [pool]
-

Invariants

- The fee written to the pool always lies in [baseFee, feeCap].
- scalingFactor is either 0 or in [1e6, 1e8], and a non-zero scalingFactor implies feeCap > baseFee.
- At most one timepoint is written per block timestamp.

Issue_38	Free tick moves through empty-liquidity regions let an attacker force cap fees
Severity	Medium
Description	Every Nest fee input is the raw pre-swap globalState.tick. beforeSwap records it in the oracle and prices the swap from its distance to the TWAP: *_writeTimepoint(tick);* *_updateDeviationFee(tick, fee);* However, the tick only means something where there is liquidity. With zero active liquidity, the required input is 0, so the swap loop moves the price to the next initialized tick or to the limit without consuming input or charging a fee: *} while (amountRequired != 0 && currentPrice != limitSqrtPrice);* So when the price sits at the edge of all liquidity, an attacker can swap 1 wei with a far limit and move the tick up to ±887,271 for gas only. The swap starts at the TWAP, so it pays only baseFee. The attacker parks the price at X and moves it back for free N seconds later. The return swap records X for the whole parked time, which shifts the TWAP by about N · (X - M) / twapWindow ticks (M is the real price). Nobody has a reason to move the price back earlier, since there is no liquidity to arbitrage. Every swap then pays baseFee plus the slope times this shift, up to feeCap, until the parked seconds leave the window, which can be up to 12 hours. The same trick just before one victim's swap raises only that victim's fee, and in MEV mode, parking across a block boundary sets _blockStartTick and saturates the LVR term for the next block. For example, take liquidity only on [-6000, 6000], the price on the upper edge, a 600-second window, a 0.05% base fee and a 1% cap. Parking near the maximum tick shifts the TWAP by about 1,470 ticks per second. So about 7 parked seconds push every swap to the 1% cap for the next 600 seconds, even at the minimum allowed slope of 1 fee unit per tick, and one second is enough from a slope of about 6.5. With a 12-hour

	window, the shift is about 20 ticks per second, so the attacker needs from 5 seconds (maximum slope) to about 8 minutes (minimum slope). This condition is common on concentrated, stable-pair, or ALM-managed pools whose price has left the range, and on a new pool before its first mint. An attacker who is also an LP in range collects the excess fees. Slippage limits don't protect users, since quotes already include the inflated fee. Each swap's loss is bounded by feeCap - baseFee.
Recommendations	Consider reading the pool's active liquidity in beforeSwap. When it is zero or below a threshold, use the first liquid tick in the swap direction as the fee and oracle tick, and apply the same rule to _blockStartTick. Avoid walking ticks, since that is unbounded. Otherwise, document and acknowledge this issue.
Comments / Resolution	

Issue_39	Own-TWAP surcharge lets farmed-range tick push persist
Severity	**Low**
Description	In TWAP-only mode each swap is surcharged on its pre-swap distance from the pool's own TWAP, so after a push the restoring trades pay the displacement surcharge until the TWAP absorbs it. With slow restoring arbitrage and rich farming rewards, a farmer could hold the tick in a narrow farmed range for up to one TWAP window per push, earning rewards even though the true price is outside the range.
Recommendations	Consider documenting and acknowledging this issue.
Comments / Resolution	

Issue_40	Wrong plugin configuration blocks swaps, and replacing the plugin freezes the fee
Severity	Low
Description	On every swap, the plugin computes the new fee and calls pool.setFee if it changed. The pool only accepts this call if the DYNAMIC_FEE flag is enabled. However, setPluginConfig accepts any value, so if the configuration keeps the BEFORE_SWAP hook but removes DYNAMIC_FEE, every swap reverts as soon as the fee changes. Also, setPlugin resets the configuration to zero. After a plugin change, the pool keeps the last dynamic fee, up to 5%, without any hook, until the configuration and the fee are set again.
Recommendations	Consider skipping the fee update when DYNAMIC_FEE is disabled, and updating the plugin, its configuration and the fee in the same transaction.
Comments / Resolution	

Issue_41	Adding liquidity around a sandwich removes the protection
Severity	Low
Description	The deviation fee aims to make sandwich attacks not profitable, by making the victim and the back-run pay an extra fee based on the distance between the price and the TWAP. However, this extra fee is paid to the liquidity in range, and adding or removing liquidity doesn't change the fee. An attacker can add liquidity before the sandwich and remove it after, and get back most of the extra fee. With the maximum slope, a sandwich that loses money without this becomes almost as profitable as on a pool with a fixed fee, and the victim loses as much as without any protection.
Recommendations	Consider documenting this limitation, or sending the extra fee to the protocol instead of the liquidity added in the same block.
Comments / Resolution	

Issue_42	Plugin config desync after a reattach silently stops incentive updates
Severity	Low
Description	The plugin keeps its own incentive and the pool's pluginConfig, whose AFTER_SWAP flag makes swaps call the incentive's crossTo. Only _updatePluginConfigInPool reconciles them. However, AlgebraPool.setPlugin always resets the config: *_setPluginConfig(0);* A pools administrator, or the Nest factory owner through NestFeePluginFactory.setPluginConfig, can also write a config without AFTER_SWAP. Then afterSwap never runs while incentive is still set, and setIncentive cannot fix it: *require(currentIncentive != newIncentive, 'Already active');* Recovery needs a disconnect and reconnect, or an administrator config write. Meanwhile, the virtual pool keeps emitting to the stale tick, so rewards can go to the wrong farmers. If the price later moved the opposite way, the first mismatched crossTo sets deactivated = true in EternalVirtualPool, which stops the incentive's emissions for good. It can then only be wound down manually.
Recommendations	Consider adding a permissioned syncPluginConfig() that calls _updatePluginConfigInPool(), and documenting the recovery sequence after a reattach. Otherwise, acknowledge this issue.
Comments / Resolution	

Issue_43	A re-attached plugin resumes its oracle across an unrecorded gap
Severity	Low
Description	AlgebraPool.setPlugin clears the plugin config. When a previously used Nest plugin is attached again, it keeps its old timepoints and _scalingFactor, while swaps run without hooks until a privileged party restores the config. When hooks return, the first recorded swap credits its tick for the whole gap: *last.tickCumulative += int56(tick) * int56(uint56(delta));* However, readiness only checks the age of the oldest timepoint, not whether the history is continuous: *return _blockTimestamp() - timepoints[oldestIndex].blockTimestamp >= window;* So the old slope is live immediately, and the TWAP is built from a tick that was never held. An attacker can move the price while hooks are off, then swap back right after the config is restored. The gap makes the TWAP equal the pre-swap tick, so the attacker pays only baseFee, while honest swaps pay up to feeCap for one twapWindow. Also, _isCurrentPlugin does not check that the pool routes BEFORE_SWAP to the plugin, so getTwapTick, getCurrentFee, and isTwapReady report values the pool is not using. The extra fees go to LPs, and this needs a non-atomic privileged re-enable.
Recommendations	Consider adding a permissioned resume() that writes a timepoint and stores a historyStart used by isTwapReady. Also consider requiring BEFORE_SWAP_FLAG in _requireActiveRecorder, isTwapReady, and getCurrentFee, and batching setPlugin with the config restore.
Comments / Resolution	

Issue_44	Fee views run while the pool is locked
Severity	Informational
Description	A swap sets its fee before the price moves, then locks the pool and asks the caller to pay. getCurrentFee can still be called during that payment. It reads the tick after the move and returns a new fee. The swap keeps the fee that was stored at the start. _getPoolState reads the price, tick, fee, and plugin config from the pool, and it discards unlocked. getCurrentFee uses that helper and does not check the lock. The pool is locked from the start of the swap through the payment callback, and by the time the callback runs the stored tick is already the post-swap tick. beforeSwap is the step that matters for the charge. It reads the pre-swap tick, computes the fee, and stores it on the pool. The swap math then uses that stored fee for the whole trade. A router, token, or other callback that calls getCurrentFee while paying sees the fee for the post-swap tick instead.
Recommendations	While the pool is locked, return the fee already stored on the pool, or make the view revert.
Comments / Resolution	

Issue_45	prepayTimepointsStorageSlots cannot prepay the ring buffer's final slot
Severity	**Informational**
Description	The timepoint buffer has 65,536 slots, but the bound check in prepayTimepointsStorageSlots limits the exclusive endpoint to 65,535: *require(amount > 0 && type(uint16).max - startIndex >= amount);* So slot 65,535 can never be prepaid. Normal oracle writes still initialize it, so only the gas prepayment is incomplete. The MAX_TWAP_WINDOW comment also says "65535 slots" for a 65,536-slot buffer.
Recommendations	Consider computing the endpoint in uint256 and allowing startIndex + amount <= 65,536, and fixing the comment.
Comments / Resolution	

NestMEVFeePlugin

The NestMEVFeePlugin contract extends NestTWAPFeePlugin with an optional fee term referenced to the HyperCore spot oracle. The term charges for LVR when the pool tick drifts from the HyperCore spot price. While klvr is zero, the plugin behaves exactly like NestTWAPFeePlugin and makes no precompile calls. When klvr is non-zero (armed), every swap reads the spot oracle once.

setOracleConfig sets klvr, the deadband, the short TWAP window, and the oracle identity. Arming requires fee headroom, a minimum scalingFactor, a fallback slope, a ready short TWAP, and a live oracle read within maxConfigGap ticks of the pool tick. The oracle identity cannot change while the plugin is armed. setFallbackScalingFactor sets the slope used when the oracle read is invalid, and it cannot be cleared while armed.

Appendix: Armed fee formula

With a valid oracle read, gap = max(|tick − spot|, |blockStartTick − spot|), capped at MAX_PRICED_GAP (10000). lvrTerm = klvr × max(0, gap − deadband) / 1e6 and sandTerm = scalingFactor × |tick − TWAP(sandWindow)| / 1e6. The fee is min(baseFee + max(sandTerm, lvrTerm), feeCap). With an invalid read, the fee is baseFee + fallbackSlope × |tick − TWAP(twapWindow)| / 1e6, also capped.

Privileged Functions

-
- setOracleConfig [plugin factory, DEVIATION_FEE_MANAGER or Algebra factory owner]
- setFallbackScalingFactor [plugin factory, DEVIATION_FEE_MANAGER or Algebra factory owner]
- setDeviationFeeConfig [plugin factory, DEVIATION_FEE_MANAGER or Algebra factory owner]

Invariants

- klvr is either 0 or in [1e6, 1e8], deadband is at most 100, and priceExp is at most 60.
- The oracle identity (spotIndex, priceExp, inverted) cannot change while klvr is non-zero.
- The priced gap never exceeds MAX_PRICED_GAP, and the final fee always lies in [baseFee, feeCap].

Issue_46	LVR fee can freeze pools away from the oracle price
Severity	Low
Description	The MEV plugin charges an extra fee based on the distance between the pool price and the HyperCore price, minus a deadband. When the oracle is enabled, the plugin checks that the deadband leaves a profit to arbitrageurs moving the price back, assuming that each tick of fee costs one tick of price. However, the fee is taken on the input amount, so a fee f costs more than f in price, and the difference grows with the fee. Around a 3% fee, arbitrageurs can't make any profit by moving the price back. The pool then stays away from the market price, and every swap pays a fee close to the 5% cap. The block start price used by the plugin also prevents arbitrageurs from moving the price back in small steps within a block. With the maximum LVR slope, a deadband of 7 ticks freezes the pool after a 2.5% move on HyperCore, and a deadband of 15 ticks after a 5% move. An attacker can also freeze the pool without any HyperCore move. One swap moving the pool by 4% only pays the base fee, since the fee is computed before the swap. The pool stays frozen with a 3.95% fee after 600 blocks of arbitrage attempts.
Recommendations	Consider limiting the LVR slope under 0.95. With a lower slope, the profit left to arbitrageurs grows with the gap, so large gaps are closed quickly. Consider computing the LVR fee from the price ratio instead of the tick distance, or requiring a deadband big enough to keep a profit up to the fee cap.
Comments / Resolution	

Issue_47	Block start gap charges too much after an arbitrage
Severity	Low
Description	To prevent manipulation within a block, the plugin charges the LVR fee on the largest of the current gap and the gap at the start of the block. However, after an arbitrageur moves the price back to the oracle price, every following swap in the same block still pays the fee of the first gap. For example, a small swap paid a 4.95% fee right after the price was fixed, against 0.15% in the next block. Also, a small swap at the start of a block can be used to apply this high fee to every following swap.
Recommendations	Consider only using the block start gap for swaps moving the price away from the oracle price, and resetting it when the oracle is disabled.
Comments / Resolution	

Issue_48	Oracle price is always trusted once enabled
Severity	Low
Description	The plugin only compares the oracle price with the pool price when the oracle is enabled. After that, any valid price from HyperCore is used, even if the market is stopped, removed, manipulated, or if the token is not pegged to the HyperCore asset anymore. In these cases, every swap pays fees up to the cap until the oracle is disabled.
Recommendations	Consider ignoring oracle prices too far from the pool TWAP, and using the TWAP fee in that case.
Comments / Resolution	

Issue_49	The arming check cannot detect a wrong inverted flag near price 1
Severity	**Low**
Description	setOracleConfig validates the oracle orientation (inverted) and scale (priceExp) only by comparing one oracle reading with the pool tick: *require(_absDiff(tick, oracleTick) <= maxConfigGap, 'Oracle gap too large');* A wrong inverted flag reads exactly -T for a true tick T, so the gap is 2|T| and the check passes whenever |T| <= maxConfigGap / 2. At MAX_CONFIG_GAP = 2000, this is any price between about 0.905 and 1.105. If the exponent is also wrong, about 17% of the price axis passes (prices near any 10^(k/2)). A feed with a constant offset up to maxConfigGap also passes. This check exists to catch exactly these mistakes. Once armed, the plugin never re-checks, so every swap pays an inflated LVR fee until the manager disarms. A pool armed at a price of about 1.10 sees a phantom gap of about 1,900 ticks, which pushes every swap to the fee cap (up to 5%). A pool armed at a price of 1 sees a phantom gap of about 200 ticks after a 1% market move.
Recommendations	Consider deriving the orientation and exponent on-chain from the spot market's tokens and the pool's decimals, or arming in two steps. Keep maxConfigGap close to the deadband.
Comments / Resolution	

Issue_50	Classic sandwiches remain profitable in several configurations
Severity	**Informational**
Description	While the spot read is live, NestMEVFeePlugin combines the sandwich term (short window _sandWindow) and the LVR term with max. The long _twapWindow is used only on the fallback path. However, the slope can be set too low to stop a sandwich: an armed plugin accepts a slope as low as 0.01, and the fallback slope is capped at 0.5: *uint64 public constant MIN_ORACLE_LIVE_SCALING = uint64(SCALING_PRECISION);* *uint64 public constant MAX_FALLBACK_SCALING_FACTOR = uint64(50 * SCALING_PRECISION);* A sandwich is profitable once the victim's own impact exceeds 2·base / (1 – s) ticks, which is about 10, 20, 50 and 100 ticks at slopes 0.01, 0.5, 0.8 and 0.9 with a 0.05% base. The LVR term (klvr · (gap - deadband)) also taxes the back-run, so these figures hold when klvr is low or the victim's impact stays inside the deadband, which can be up to 100 ticks. On the fallback, no configuration blocks a sandwich of a victim that moves the price by more than about 4·base ticks. Even at slope 1.0, the fee cap limits protection: victims whose impact exceeds (baseFee + feeCap) / 100 ticks (about 105 with a 0.05% base and a 1% cap) can still be sandwiched. Arming also switches the pool from the long TWAP to the short one, which a held price pulls much faster.
Recommendations	Consider documenting the region the plugin actually protects, raising MIN_ORACLE_LIVE_SCALING, and allowing the fallback slope up to MAX_SCALING_FACTOR.
Comments / Resolution	

Issue_51	Anyone can make the arming call setOracleConfig revert by moving the pool just before it
Severity	Informational
Description	NestMEVFeePlugin.setOracleConfig requires the pool tick to be within maxConfigGap of the HyperCore spot tick: *require(_absDiff(tick, oracleTick) <= maxConfigGap, 'Oracle gap too large');* The check uses the instantaneous pool tick, so an observer can swap a thin pool beyond the gap just before the manager's transaction and make it revert. The attacker pays fees and price impact, and the manager can retry, so the effect is only a delay.
Recommendations	Consider sending the arming transaction privately, or batching a corrective swap with it.
Comments / Resolution	

Issue_52	Disarming requires echoing the current feed identity
Severity	**Informational**
Description	NestMEVFeePlugin.setOracleConfig both arms and disarms (klvr == 0). While armed, it requires the feed identity to stay unchanged: *require(spotIndex == _oracleSpotIndex && priceExp == _oraclePriceExp && inverted == _oracleInverted, 'Disarm before identity change');* However, this also applies to a disarm. So setOracleConfig(0, 0, 0, 0, 0, false, 0) reverts unless the live identity is zeros, and during a feed incident the manager must first read mevFeeConfig() and pass the identity back. Also, a successful disarm stores all given parameters without validation, such as shortTwapWindow, so mevFeeConfig() can return meaningless values.
Recommendations	Consider skipping the identity check when klvr == 0, and writing only _klvr = 0 on disarm or adding a dedicated disarm().
Comments / Resolution	

Issue_53	Code-quality notes in NestMEVFeePlugin
Severity	**Informational**
Description	None of these put funds at risk: - The pluginFactory branch in setOracleConfig and setFallbackScalingFactor is dead, since NestFeePluginFactory never calls them. - MIN_ORACLE_LIVE_SCALING equals the floor the parent already enforces, so it only enforces a non-zero slope. - The feeCap > baseFee and price != 0 checks in setOracleConfig are already implied by earlier checks. - The armed checks in setDeviationFeeConfig run before the access check, so an unauthorized caller gets a misleading revert reason. - _lastOracleLive has no getter, is not reset on disarm, and is set on arming without emitting OracleFallback(false). - The _blockStartBlock == uint32(block.number) check in _feeFromObservation is always true on the swap path. - sandTerm runs a TWAP binary search even when lvrTerm alone already reaches feeCap. - __mevGap is uint256[49], which gives 51 slots instead of the usual 50. - The _fallbackScalingFactor != 0 and scalingFactor != 0 fallbacks and the window == 0 return are unreachable while armed, and only matter after a beacon round trip.
Recommendations	Fix as required.
Comments / Resolution	

NestMEVFeePluginFactory

The NestMEVFeePluginFactory contract is a concrete NestFeePluginFactory with no added logic. It deploys beacon-proxy NestMEVFeePlugin instances for classic, existing, and custom pools. Plugins start disarmed, and a fee manager arms the oracle term on each plugin afterwards.

HyperCoreSpotOracle

The HyperCoreSpotOracle library reads a HyperCore spot price from the HyperEVM precompile at 0x0808 and converts it to an Algebra tick. Any failure comes back as available = false instead of reverting.

getOracleTick makes a staticcall with a fixed 20000 gas stipend and rejects the result unless the call succeeded with exactly 32 bytes of return data. tryConvertPriceToTick turns priceRaw / 10^priceExp into a tick, inverting the ratio when the inverted flag is set. It rejects zero or oversized prices, oversized exponents, and square-root prices outside the TickMath range.

Invariants

- available is true only if the precompile call succeeded with exactly 32 bytes of return data and every conversion bound passed.

Issue_54	Price conversion doesn't work for some token decimals
Severity	**Medium**
Description	To convert the HyperCore price into a pool price, the library divides it by 10^priceExp, or divides 10^priceExp by it when inverted. However, some pairs need to multiply by a power of 10 instead, depending on the token decimals and the HyperCore market. This is the case for an 8 decimals or a 6 decimals token against WHYPE. The oracle can never be enabled for these pairs. Also, the inverted conversion rejects any exponent above 19, while MAX_ORACLE_PRICE_EXP is 60 and the math would work. Raw prices above 2^64 - 1 are also rejected, so the MAX_RAW_PRICE limit can never be reached.
Recommendations	Consider supporting negative exponents, and documenting which decimals are supported and which spot pair index should be used.
Comments / Resolution	

Issue_55	Coarse HyperCore prices make the oracle tick jump by up to the deadband
Severity	**Low**
Description	HyperCoreSpotOracle.tryConvertPriceToTick converts the raw integer spotPx to a tick, but only rejects zero or oversized values: *if (priceRaw == 0 || priceRaw > MAX_RAW_PRICE) return (0, false);* One raw unit is a relative change of 1 / priceRaw, so the tick moves in steps of about 10^4 / priceRaw: about 20 ticks at a raw price of 500, and about 100 (MAX_DEADBAND) at 100. Neither arming nor swaps check the resolution, so the gap NestMEVFeePlugin compares with the deadband can be off by up to one step. The LVR term can then charge a phantom fee or hide a real gap. Fees stay capped, and no funds are at risk.
Recommendations	Consider requiring a minimum priceRaw (for example 1e5) when arming, or documenting that the deadband must be at least 10^4 / priceRaw ticks.
Comments / Resolution	

Farming

Issue_56	Farming can't support custom pools and both old and new classic pools
Severity	**Low**
Description	The farming contracts are out of scope, but they are affected by the deployer change. Before adding or removing an incentive, FarmingCenter checks that the pool address is equal to the address computed from the position manager pool deployer, using the classic salt. This doesn't work for every pool: - custom pools never pass, since their address also depends on the custom deployer; - the position manager only has one deployer, so either the pools created before the upgrade or the ones created after never pass. If the current farming center is kept, it still uses the old position manager. It only supports old pools, and positions created with the new position manager can't be farmed at all.
Recommendations	Consider checking pools with the factory mappings instead.
Comments / Resolution	